

Department of Computer Science & Engineering

Course Code: CSL37

TERM: Oct 2024 – Jan 2025

Course Name: **Object Oriented
 Programing Laboratory**

Faculty In-charge: **Jamuna S Murthy**

Credits: **0:0:1**

SEMESTER : **III**

Design, develop, and implement the following programs in Java

PART - A		CO	PO/PSO
1.	Design and implement a flight booking system using inheritance, showcasing different types of constructors. The system should include a base class Flight and two derived classes: DomesticFlight and InternationalFlight. 1. Base Class Flight: ○ Attributes: String flightNumber, String airline, double basePrice. ○ Constructors: Use Default and parameterized. ○ Method: void displayDetails() to print flight details. 2. Derived Class DomesticFlight: ○ Attribute: double domesticTaxRate. ○ Constructors: Use super to call base class constructors and add a custom constructor. ○ Methods: ■ Override displayDetails() to include the total price. ■ double calculatePrice() computes total price. 3. Derived Class InternationalFlight: ○ Attributes: double internationalTaxRate, double customsFee. ○ Constructors: Use super and add a custom constructor. ○ Methods: ■ Override displayDetails() to include the total price. ■ double calculatePrice() computes total price. 4. Main Method: ○ Create objects using different constructors. ○ Use dynamic dispatch to call displayDetails() for both classes.	1	2,3,5/3
2.	Design a Java application for a car rental service using interfaces, focusing on objects as parameters, argument passing, and returning objects. The application should allow users to: 1. Define an Interface RentalService:	1	2,3,5/3

	○ Methods: checkAvailability(), rent(), returnCar(), and calculateRentalCost(int days, double distance). 2. Create a Class Car Implementing RentalService: ○ Attributes: make, model, year, dailyRate, isAvailable. ○ Methods: ■ Constructor to initialize attributes. ■ Implement interface methods. ■ Add displayDetails() to show car details. ■ Add compareRates(Car other) to return the car with the lower rate. 3. Create a Class CarRentalService: ○ Attributes: A list of Car objects. ○ Methods: ■ addCar(Car car): Adds a car to the service. ■ findCar(String make, String model): Returns a car based on make and model. ■ displayAvailableCars(): Shows all available cars. ■ Menu-based options: Check availability, rent a car, return a car, compare rates, and calculate rental cost. 4. Main Method: Demonstrate the use of objects as parameters, argument passing, and returning objects through a menu-based system for the car rental service.		
3.	Create a program to simulate a weather monitoring system using inheritance, dynamic method dispatch, and String and StringBuffer handling. The program should have a base class WeatherStation and two derived classes: TemperatureStation and RainfallStation. 1. Base Class: WeatherStation ○ Attributes: String location, String stationId ○ Method: void displayData(): Print generic weather station data. Add methods demonstrating String handling, such as concatenation, substring extraction, and comparison. 2. Derived Class: TemperatureStation ○ Attribute: double temperature ○ Methods: Override displayData() to include temperature details. Add a method demonstrating StringBuffer usage (e.g., appending, replacing, and reversing a string). 3. Derived Class: RainfallStation ○ Attribute: double rainfall ○ Methods: Override displayData() to include rainfall data. Add a method demonstrating another aspect of StringBuffer handling (e.g., inserting, deleting, or replacing). 4. Main Method: ○ Use an array of WeatherStation references to store objects of both TemperatureStation and RainfallStation.	1	2,3,5/3

	○ Call displayData() dynamically for each object. ○ Demonstrate String and StringBuffer handling explicitly by: Creating a String to represent the weather station's name and manipulating it using String methods (e.g., concatenation, case conversion, substring). Using a StringBuffer to modify station details dynamically, such as appending data or reversing station IDs.		
4.	Implement a simple restaurant ordering system using inheritance, demonstrating the usage of final and super. The program should consist of a base class MenuItem and two derived classes: FoodItem and DrinkItem. 1. Create a base class MenuItem with the following: ○ Attributes: final String name and double price. ○ Constructor: Initialize name and price attributes. ○ Method: void displayDetails(): Prints basic menu item details. Mark the displayDetails() method as final to prevent overriding. 2. Create a derived class FoodItem with the following: ○ Attribute: boolean isVegetarian. ○ Constructor: Initialize name, price, and isVegetarian attributes. ○ Method: Add a displayFoodDetails() method that calls the super.displayDetails() method and includes whether the food item is vegetarian. 3. Create another derived class DrinkItem with the following: ○ Attribute: boolean isAlcoholic. ○ Constructor: Initialize name, price, and isAlcoholic attributes. ○ Method: Add a displayDrinkDetails() method that calls the super.displayDetails() method and includes whether the drink is alcoholic. 4. In the main method Create an array of MenuItem references and store objects of both FoodItem and DrinkItem. Use dynamic method dispatch to call the displayDetails() method for each item. Demonstrate the use of final and super in the overridden methods.	1	2,3,5/3
5.	Design and implement a vehicle management system using inheritance, overloading, and static methods. The program should have a base class Vehicle and two derived classes: Car and Truck. The system should allow the user to manage and display information about different types of vehicles. A. Create a base class Vehicle with the following: a. Attributes: String make, String model, and int year. b. Methods: i. void displayDetails(): Displays basic vehicle information (make, model, year). ii. Overload displayDetails to accept a boolean parameter detailed. If detailed is true, display additional vehicle-specific information.	1	2,3,5/3

	iii. Add a static method void showType() to display the type of class (e.g., "Vehicle"). B. Create a derived class Car with the following: a. Attribute: int numberOfDoors. b. Methods: i. Override displayDetails() to include the number of doors. ii. Overload displayDetails to display details with a custom format. iii. Add a static method void showType() to display the type of class (e.g., "Car"). C. Create another derived class Truck with the following: a. Attribute: double loadCapacity (in tons). b. Methods: i. Override displayDetails() to include the load capacity. ii. Overload displayDetails to display details with a custom format. iii. Add a static method void showType() to display the type of class (e.g., "Truck"). D. In the main method Create an array of Vehicle references and store objects of both Car and Truck. Use dynamic method dispatch to call the displayDetails() method for each object, displaying the details specific to the type of vehicle. Call the static method showType() on each class to demonstrate its usage.		
6.	Create an online shopping Class with the following requirements demonstrating the usage of packages and customized exception handling: A. Create a package named shopping. B. Define an interface Product with methods getPrice(), getDetails(), and printDetails(). C. Create a base class Item that implements Product with common properties like productId, productName, and price. D. Create two subclasses: a. Electronics: Adds properties like warranty and brand. Override getDetails() to include these. b. Clothing: Adds properties like size and material. Override getDetails() to include these. E. Use another class Cart that allows adding products and calculates the total price of items. F. Handle exceptions: a. ProductNotAvailableException if the product added to the cart is not in stock. b. InvalidProductException if an invalid product is added. G. Demonstrate the functionality with a main() method in a package application, ensuring all features are tested.	2	2,3,5/3

7.	Design and implement a University Management System demonstrating abstract classes and interfaces with the following features: A. Create a Package university: a. Interface Person: Methods: getDetails() and printDetails(). B. Create a Base Class Employee: a. Implements the Person interface. b. Attributes: name (String): Employee's name, age (int): Employee's age, designation (String): Employee's job title, salary (double): Employee's salary. C. Create Two Subclasses: a. Professor: i. Additional attributes: department (String): Department, researchArea (String): Research area. ii. Override getDetails() and printDetails() to include the new attributes. b. AdminStaff: i. Additional attributes: role (String): Role in the administration, workingHours (int): Number of working hours. ii. Override getDetails() and printDetails() to include the new attributes. D. Main Method: Create objects for Professor and AdminStaff. Prompt the user to enter details for each object. Display the entered details for both Professor and AdminStaff.	1	2,3,5/3
8.	Design a Java banking application demonstrating System-defined Exceptions, and User Defined Exception handling, with the following requirements: A. Create a Package banking: ● Define an Interface Account: ○ Methods: void deposit(double amount), void withdraw(double amount), double checkBalance() ● Implement the Interface in a Class BankAccount: ○ Common Properties: String accountNumber, String accountHolder, double balance ○ Constructor: Initialize account details and balance. ○ Methods: Implement the methods from Account. ● Create Subclasses of BankAccount: ○ SavingsAccount: ■ Additional Property: double interestRate. ■ Method: double calculateInterest(): Calculate interest using the formula: $\text{Interest} = \text{balance} \times \text{interestRate} \times \text{timePeriod}$ ■ Override withdraw() to throw a custom InsufficientFundsException if the withdrawal amount exceeds the balance.	2	2,3,5/3

	○ CurrentAccount: ■ Additional Property: double overdraftLimit. ■ Override withdraw() to throw a custom OverdraftLimitExceededException if the withdrawal amount exceeds the balance and overdraft limit combined. B. Handle Exceptions: ● System-Defined Exception: Handle invalid input using NumberFormatException. ● Custom Exceptions: Define InsufficientFundsException and OverdraftLimitExceededException in the banking package. C. Main Class: Create objects for SavingsAccount and CurrentAccount. Perform operations like deposit, withdraw, and check balance. Use the calculateInterest() method for SavingsAccount and demonstrate mathematical calculations. Handle both system-defined and custom exceptions during operations.		
PART - B			
1.	Create a simple library management system where multiple users (threads) can borrow and return books at the same time by implementing Runnable Interface. A. Create a Book class: - The class should have the following attributes: bookTitle (String) - title of the book, author (String) - author of the book, availability (boolean) - a flag to indicate whether the book is available for borrowing. - Implement the following methods in the Book class: borrow() - Allows a user to borrow the book. If the book is already borrowed, it should return false. returnBook() - Marks the book as available again after returning it. B. Create a User class that implements the Runnable interface: - Each User represents a thread trying to borrow and return a book. - Each User should: Attempt to borrow a book using the borrow() method, Simulate reading the book by sleeping for a random amount of time, Return the book using the returnBook() method. C. In the main method: Create a Book object. Create multiple User threads and assign them the task of borrowing and returning the book. Start the threads and use the join() method to ensure that all threads finish their execution before the program terminates.	2	2,3,5/3
2.	Create a Java program that contains a JFrame with a JButton and a JLabel. When the button is clicked, it should change the text of the label and also change the background color of the frame. ● Use ActionListener to handle the button click event. ● The button should change the text of the label to "Button Clicked!".	3	2,3,5/3

	The background color of the frame should change to a random color each time the button is clicked.		
3.	Implement a task scheduling system where multiple tasks (threads) are executed based on their priority. A. Create an abstract class Task with methods execute() and getPriority(). B. Create two subclasses: HighPriorityTask and LowPriorityTask, each overriding the execute() method to simulate long-running tasks (e.g., printing large numbers, waiting, etc.). C. Implement a scheduling system where high-priority tasks are executed before low-priority tasks using thread priorities. D. Demonstrate the use of isAlive() to check if a task has completed and join() to wait for all tasks to finish before terminating the program.	2	2,3,5/3
4.	Create a Java program that uses a block lambda to perform an operation involving multiple steps. Specifically: - Write a functional interface StringProcessor with a method process(String input). - Implement a block lambda that: Reverses the input string, Convert it to uppercase, Returns the final modified string. - Demonstrate the block lambda by applying it to an array of strings and printing the results.	3	2,3,5/3
5.	Design a program to demonstrate synchronization of an ATM system where multiple customers (threads) can perform transactions (deposit and withdraw) on a shared ATM. A. ATM Class: - Methods: withdraw(double amount): Withdraw money. deposit(double amount): Deposit money. getBalance(): Get current balance. B. Customer Class: Each customer (thread) will perform a series of transactions: deposit, withdraw, and check balance. C. Use synchronization to ensure that only one thread accesses the ATM's balance at a time. D. Implement thread suspension, resumption, and stopping. Use isAlive() and join() to manage thread execution and ensure all customers complete their transactions.	2	2,3,5/3
6.	Create a Java program that fulfills the following requirements: - The user can draw lines on a JPanel by clicking and dragging the mouse. - As the user drags the mouse while holding the button, the program should continuously draw a line. - Use a MouseAdapter class to handle mouse events.	3	2,3,5/3
7.	Write a Java program to filter and print names from a list that start with the letter 'A' using Lambda expressions.	3	2,3,5/3
8.	Design a Java Swing-based calculator application that performs basic arithmetic operations (Addition, Subtraction, Multiplication, Division). Include a clear button to reset inputs.	3	2,3,5/3

Marks Distribution

Conduction and Result	Part	Write-up	Execution	Viva	Total	Change of Program
	A	5M	20M	7M	50	-8M
	B	3M	15M			

Course Coordinator
Jamuna S Murthy

Reviewer
J. Geetha

HoD, Dept. of CSE